NULL

Question Deeper. Know Better.

Product documentation: what NULL does, how it looks today, how it is built, and what is still missing.

VERSION EF9AD5E

31 OCTOBER 2026

Contents

1. NULL at a glance 2

2. How a conclusion is reached 3

- 2.1 The analysis pipeline 3
- 2.2 The five assessments 3
- 2.3 Where models fit 4

3. Visual language 4

4. Product tour 5

- 4.1 Sign-in 5
- 4.2 Overview 5
- 4.3 Investigations 6
- 4.4 Investigation workspace 7
- 4.5 Evidence Network 13
- 4.6 Assurance 14
- 4.7 Reports 16
- 4.8 Data Sources 17
- 4.9 Settings 18
- 4.10 Phone layout 19

5. Demonstration data 20

6. Architecture 21

7. Running NULL 23

8. Verification status 24

9. Known limits and defects 25

- 9.1 Defects found while preparing this document 25
- 9.2 Before real use 25

1. NULL at a glance

NULL tests whether a conclusion is justified by the evidence behind it. Give it a claim such as “no unauthorized access occurred”, and it asks whether the available evidence could ever have shown otherwise.

Most security reporting treats silence as good news: no alerts, so nothing happened. NULL treats silence as a question. It works out what each possible explanation predicts, checks those predictions against raw records, recommends the next check that best separates the explanations, and runs that check only after a person approves it. If the evidence exposes a weakness, NULL tracks the fix until records show that the fix worked.

> **The rule the system is built around:** a model can propose, but only deterministic code can decide. A language model may suggest explanations. It never sets a conclusion or a statistic.

Where it stands today

NULL is a working full-stack application running on synthetic data for a fictional company, Cobalt Freight. Every page and workflow in this document runs end to end. It is a demonstration, not a production service.

WORKING	SIMULATED	NOT AVAILABLE
All eight product areas	All seven data sources	Live connectors
Deterministic evidence engine	Execution of fixes: it writes records and changes no system	Live Meta model, which has never run against the real API
Sign-in, four roles, audit log	Recurrence of a fixed weakness	Personal API tokens
Reports with PDF and CSV export		Notification delivery
Remediation tracked to verification		

Three types of claim can be tested: absence of unauthorized access, monitoring completeness, and MFA enforcement. Any other claim concludes UNVERIFIED, and NULL says that it cannot test it. Section 9 lists the known limits and defects.

The screenshots here come from version `ef9ad5e`, the code currently on GitHub, running as a production build at 1440 × 900 on a fresh demo database. They use the app's only theme, which is dark.

2. How a conclusion is reached

NULL covers the life of a conclusion, from checking that it could be sound to proving that the fix behind it still holds.

1	2	3	4	5
Prevent	**Investigate**	**Mitigate**	**Verify**	**Continuous assurance**
Measure coverage and collection gaps before anyone relies on them.	Test a claim against competing explanations.	Approve and run the corrective action.	Confirm the fix from records, not from a button.	Rerun verification and catch recurrence.

2.1 The analysis pipeline

```
claim  →  compiler  →  realities (predictions over named observables)
                            ↓
raw records  →  analysis tools  →  observations  →  validator  →  assessment
                                                          ↓
                          information value ranks the next check  →  human approval
```

STEP	WHAT IT DOES
Compiler	Turns plain language into a testable claim, and states what it would take to establish it.
Realities	Competing explanations. Each declares what it predicts for each observable. They stay hypotheses even when the evidence fits.
Observables	Named questions answered from raw records with pandas, SciPy and statsmodels. Each result keeps its method, record IDs and a SHA-256 hash of those records.
Validator	Compares predictions with observations and assigns the assessment. This step is deterministic code.
Information value	The mutual information, in bits, between a check's outcome and the surviving realities, using a uniform prior and declared likelihoods. It ranks checks and is not a probability of intrusion.

2.2 The five assessments

ASSESSMENT	MEANING
SUPPORTED	Every alternative is excluded, and the claim's required predictions all matched.
CONTRADICTED	Observed evidence conflicts with what the claim requires.
PARTIALLY IDENTIFIED	Alternatives were ruled out, but part of the claim still can't be established from the evidence.
UNDERDETERMINED	More than one explanation still fits. Nothing observed yet separates them.
UNVERIFIED	There are no observations, or NULL can't test this kind of claim.

2.3 Where models fit

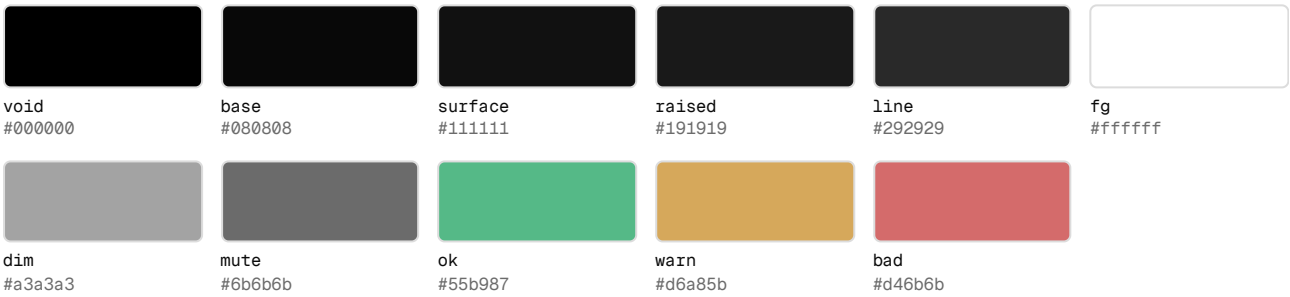
Three reasoning roles work on shared investigation state. **Explorer** looks through the evidence, **Worldbuilder** proposes realities, and **Challenger** attacks them. They use Meta's Chat Completions protocol when a key and model are configured. Otherwise, or if a call fails, rule-based roles take over, and the investigation says so and records why.

Model output is checked against a schema. References to unknown evidence or observables are dropped. A model can run only the read-only analyses already permitted, and any check that needs approval is refused. Every screenshot here shows the rule-based roles, because no key is configured.

3. Visual language

The product is monochrome and dark, and color is reserved for state. If something is green, amber or red, it means something.

Color tokens



Assessment marks

Each assessment pairs a color with a symbol, so it never depends on color alone.

```
✓ SUPPORTED          × CONTRADICTED          – PARTIALLY IDENTIFIED
○ UNDERDETERMINED    ◌ UNVERIFIED
```

Type and brand

Geist Sans sets the interface and Geist Mono sets IDs, timestamps, statistics and labels. The mark is a black circle with a white center dot, and the tagline is “Question Deeper. Know Better.” The sign-in page uses large concentric rings as its only decoration. Every signed-in page keeps a banner stating that the workspace is synthetic.

4. Product tour

This section follows one investigation from sign-in to a verified fix. The claim is the one a security lead might be asked to sign off: “Can we confirm that no unauthorized access occurred during the last 24 hours?”

4.1 Sign-in

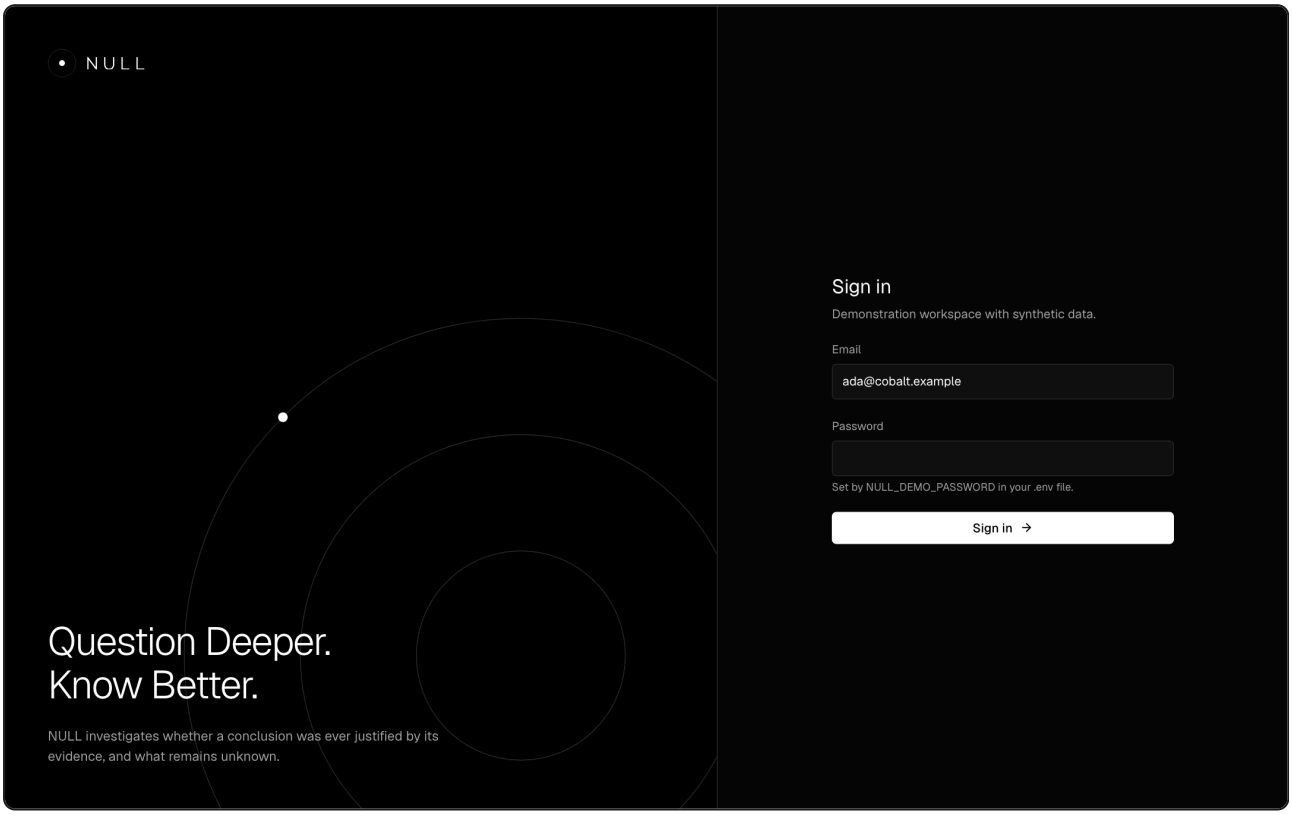


FIGURE 1 Sign-in. The page states up front that the workspace holds synthetic data.

4.2 Overview

Overview answers one question: what do we know, and what don't we? Five counters lead to the relevant page. Below them, Assurance overview shows coverage, gaps and progress, and Priority attention lists what needs a person. The sidebar footer always shows how many sources are connected and which reasoning mode is active.

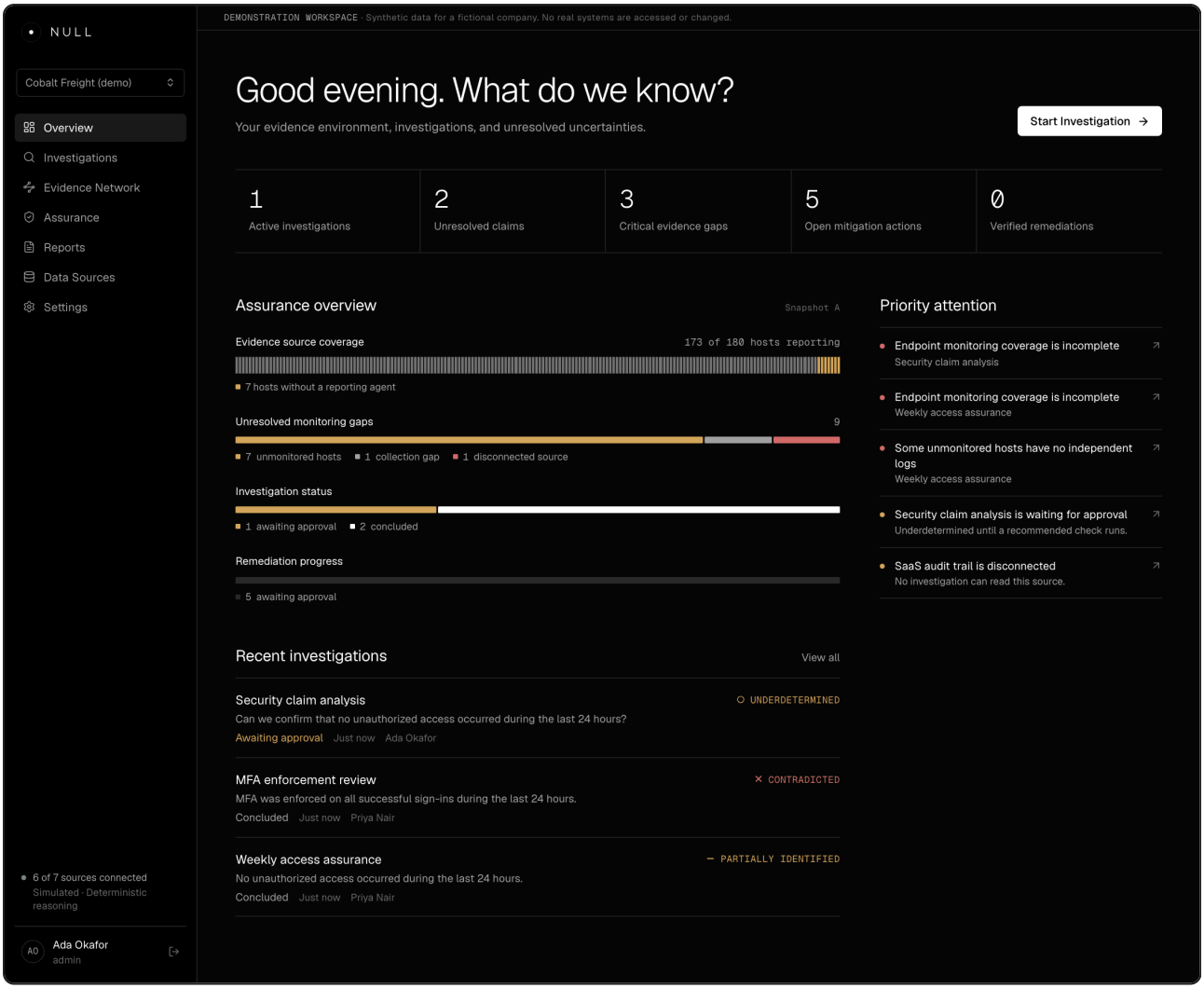


FIGURE 2 Overview on the seeded demo. 173 of 180 hosts are reporting, and one investigation awaits approval.

4.3 Investigations

The list shows each claim with its assessment and status, and can be searched and filtered by status, conclusion and date.

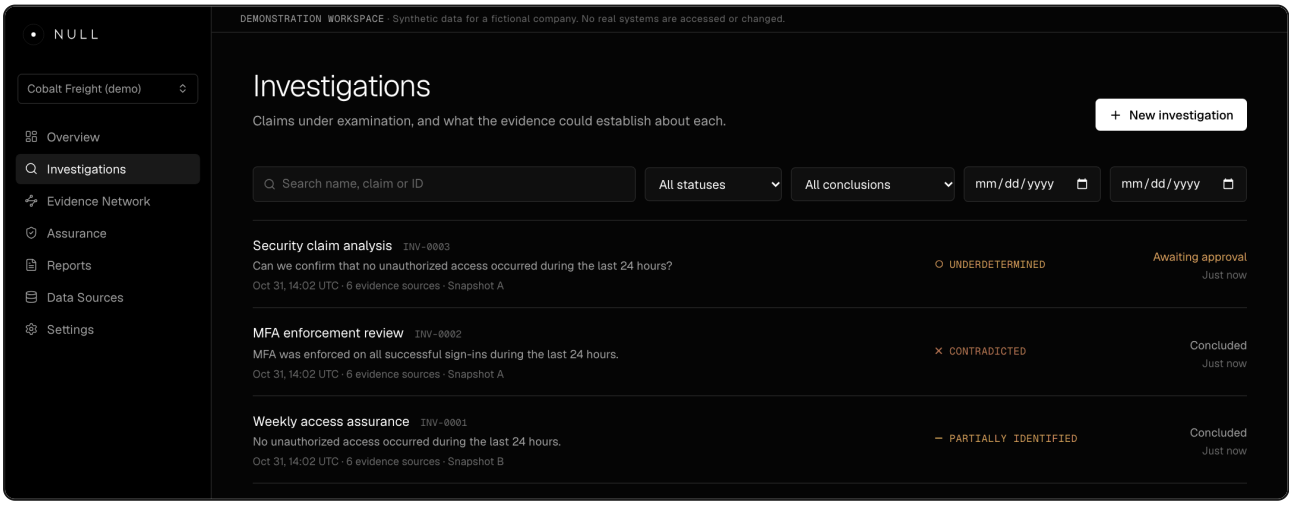


FIGURE 3 Investigations. The three seeded investigations sit at different stages.

A new investigation takes four steps: **Claim**, **Scope**, **Evidence** and **Review**. As the claim is typed, NULL shows how it reads it. A disconnected source can't be selected. The first pass only reads the sources chosen here, and anything further waits for approval.

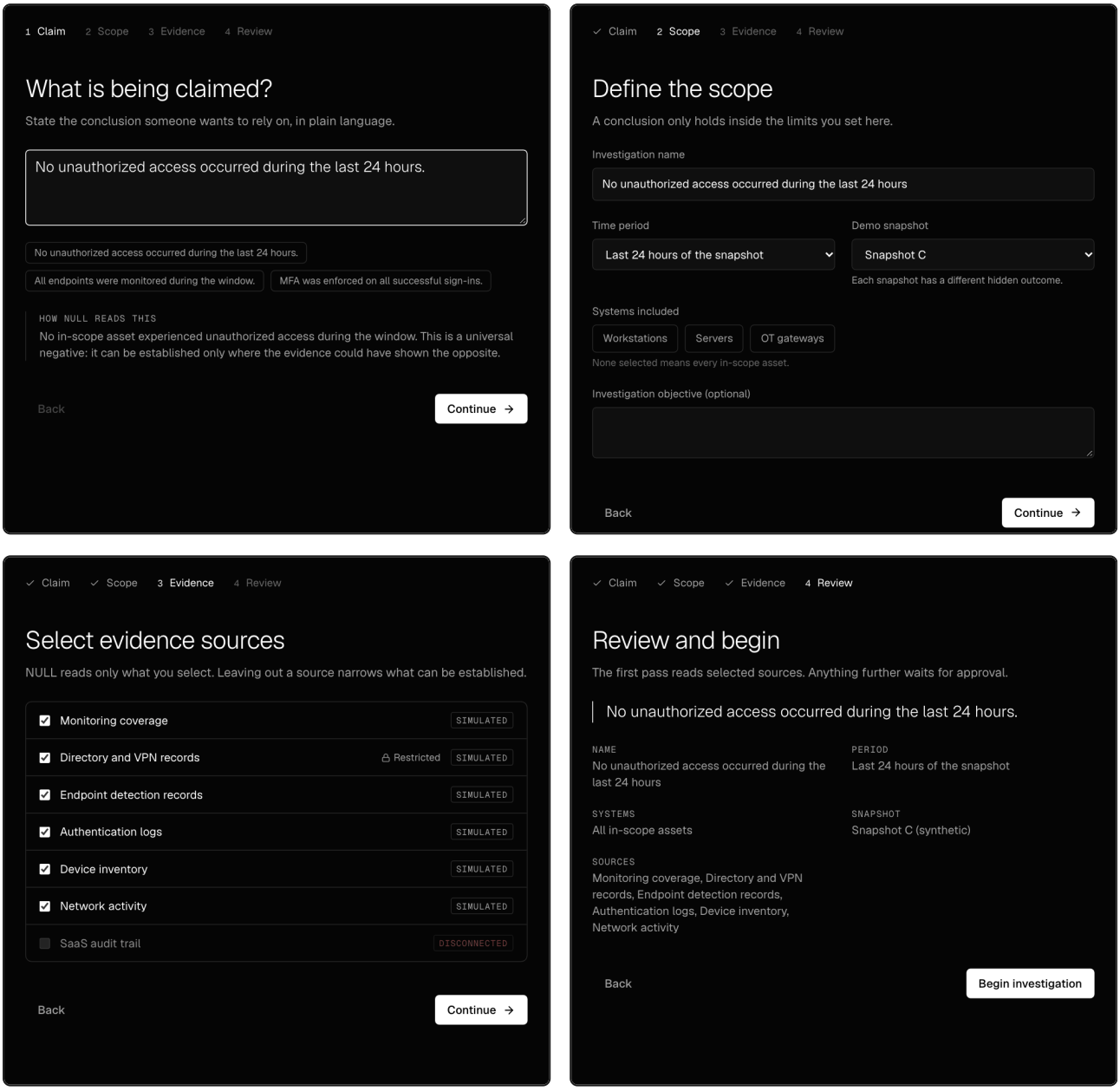


FIGURE 4 The four steps. SaaS audit trail is disconnected, so it can't be selected.

4.4 Investigation workspace

The workspace is the center of the product. The top shows the claim and how NULL interpreted it, next to the current assessment. The ledger below separates what was **observed**, **inferred** and **assumed**, what is **unknown**, and what **could change this**. The timeline on the right records each stage.

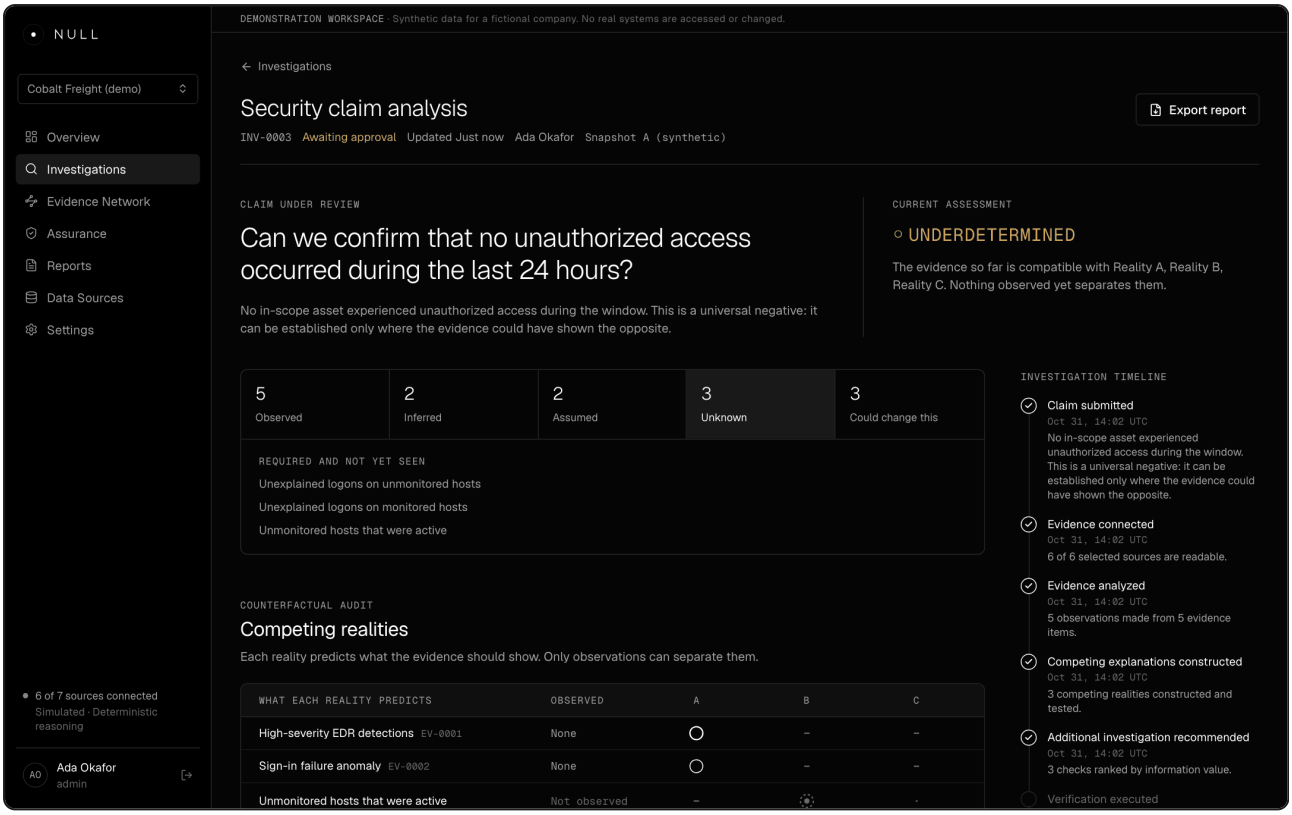


FIGURE 5 Before approval the claim is UNDERDETERMINED. EDR is quiet, but three required observations are missing.

Competing realities

The first pass produces three explanations for the same quiet evidence: no intrusion (A), an undetected intrusion on an unmonitored endpoint (B), and a detection failure on a monitored endpoint (C). The table shows what each predicts. The first two rows fit Reality A, but they can't separate it from B or C. The rows that can are still unobserved.

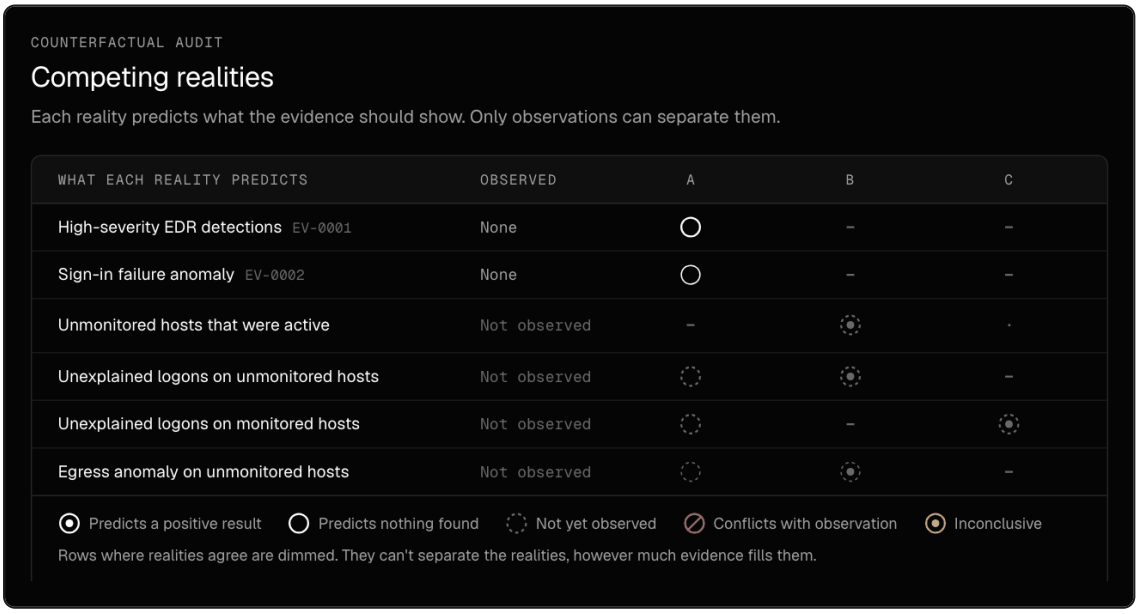


FIGURE 6 Each reality's predictions against what has been observed so far.

Recommended investigation

NULL ranks the available checks by information value and explains why the top one matters. Inspecting independent authentication records scores 0.94 bits, far above the others, because domain controllers and the VPN record logons whether or not an EDR agent is present.

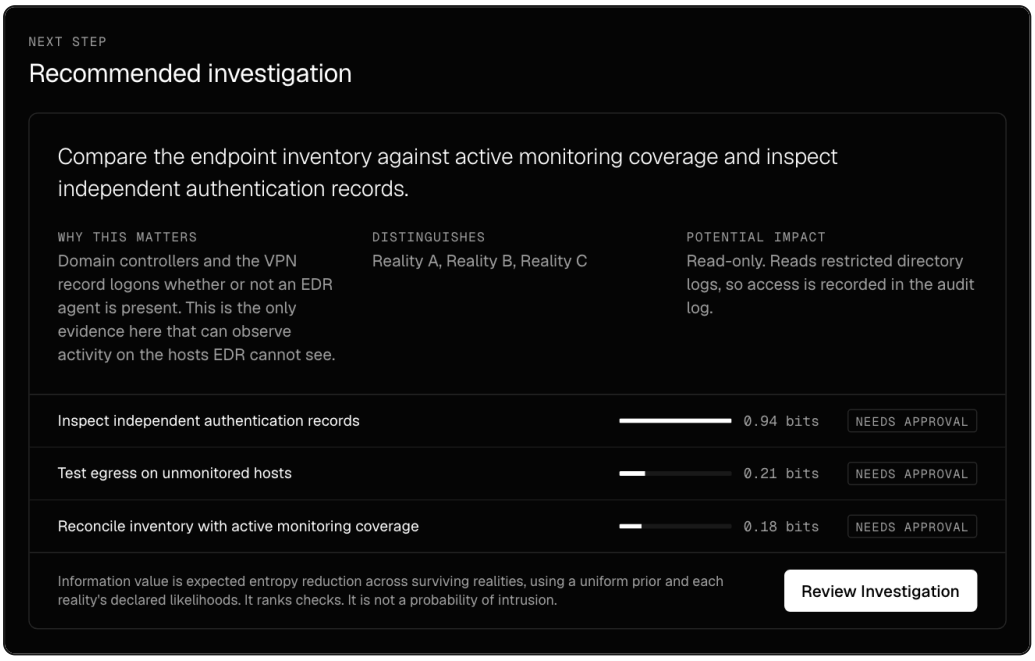


FIGURE 7 Checks ranked by information value, each marked as needing approval.

Evidence and provenance

Every evidence item names the tool that produced it, how reliable and relevant it is, and the records it rests on. Selecting an ID opens a drawer with the method, a SHA-256 hash of the cited records, and the raw records themselves.

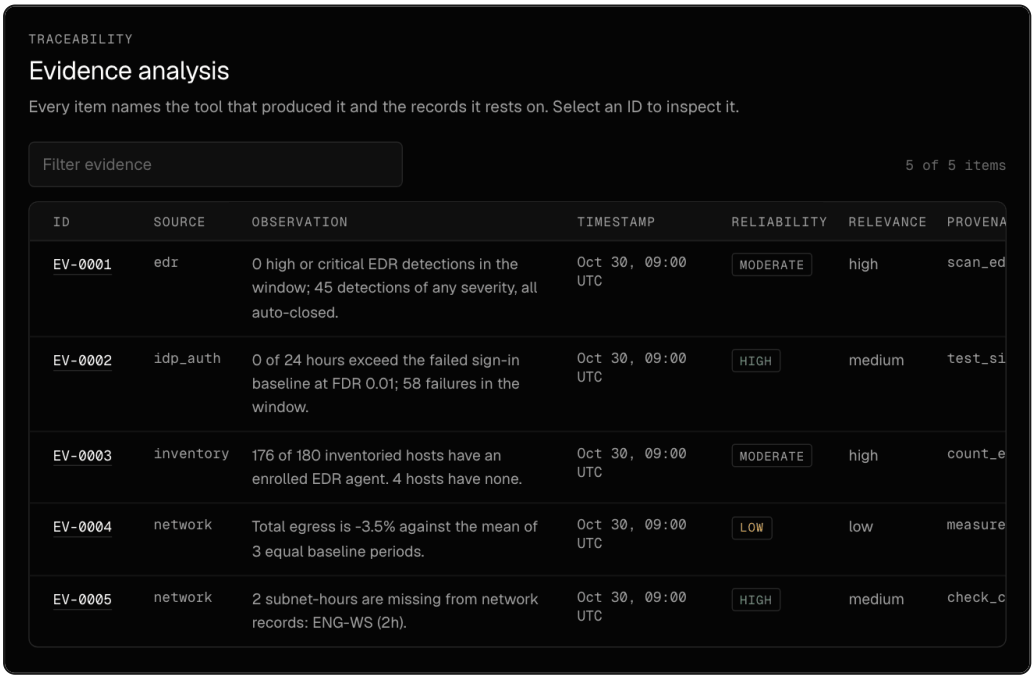


FIGURE 8 Evidence table. Provenance is the right-most column and needs a sideways scroll at this width.

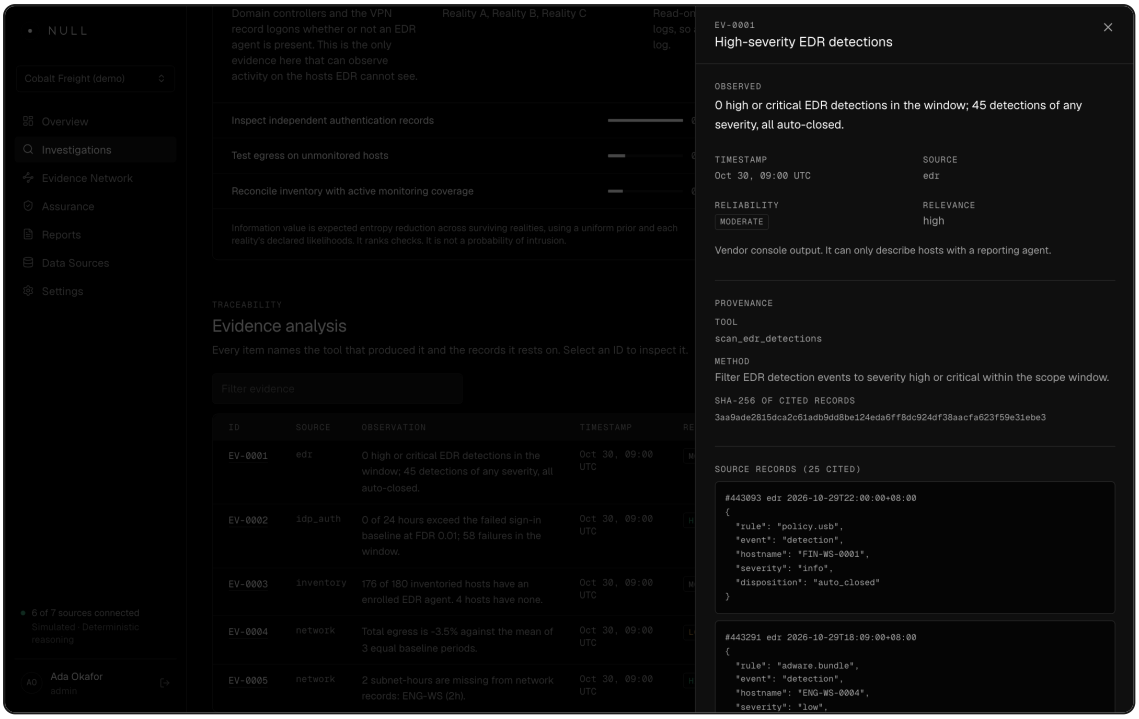


FIGURE 9 Evidence drawer for EV-0001: observation, reliability, tool, method, hash and source records.

Human approval

Nothing further runs until someone approves it. The dialog lists each procedure, its information value, the permissions it needs and its operational impact. **Approve and run** stays disabled until the person ticks the authorization box. Reads of restricted sources go to the audit log.

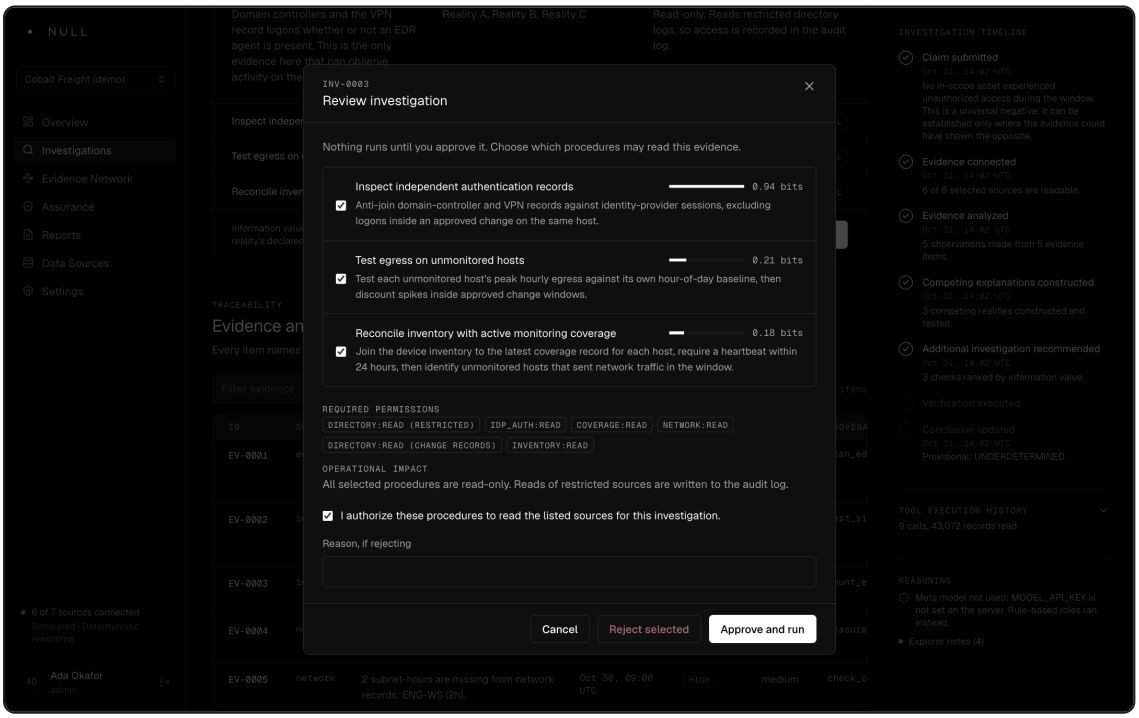


FIGURE 10 Review investigation. Procedures can be approved or rejected individually.

Conclusion

The independent logs change the picture. A service account logged on to FIN-WS-0014 with no identity-provider session and no approved change, and that host later moved far more data than its baseline. The assessment becomes CONTRADICTED. Realities A and C are excluded by their own predictions, and Reality B remains as a hypothesis.

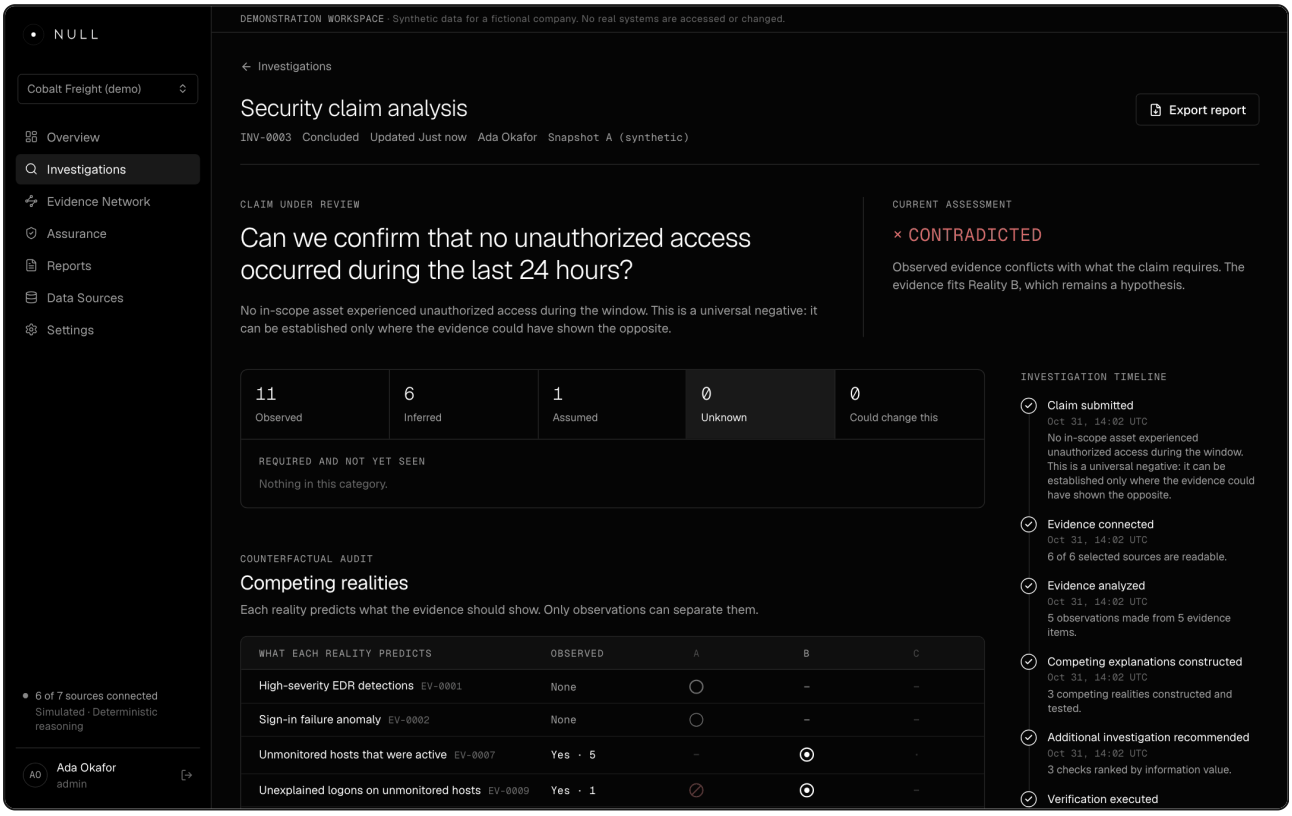


FIGURE 11 After approval the same claim is CONTRADICTED. The unknowns have dropped to zero.

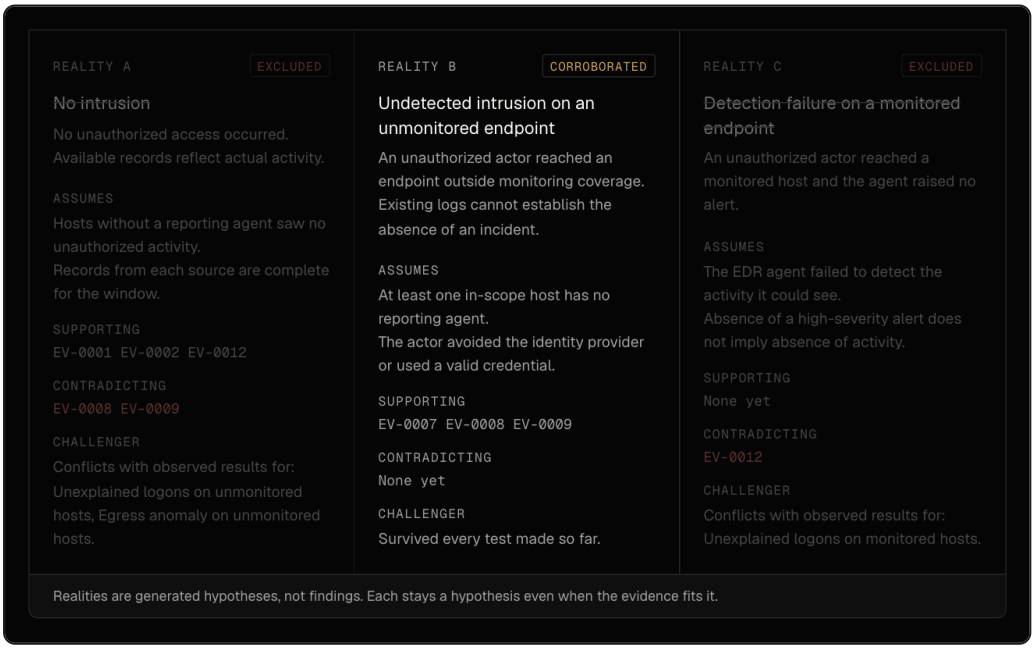


FIGURE 12 Realities A and C are excluded because observations conflict with what they predict. Reality B is corroborated.

What NULL discovered

Findings explain why the original reading was wrong. The most instructive is the gap between enrollment and reporting: three hosts have an enrolled agent that stopped reporting, so a console counting enrollment still shows them as covered.

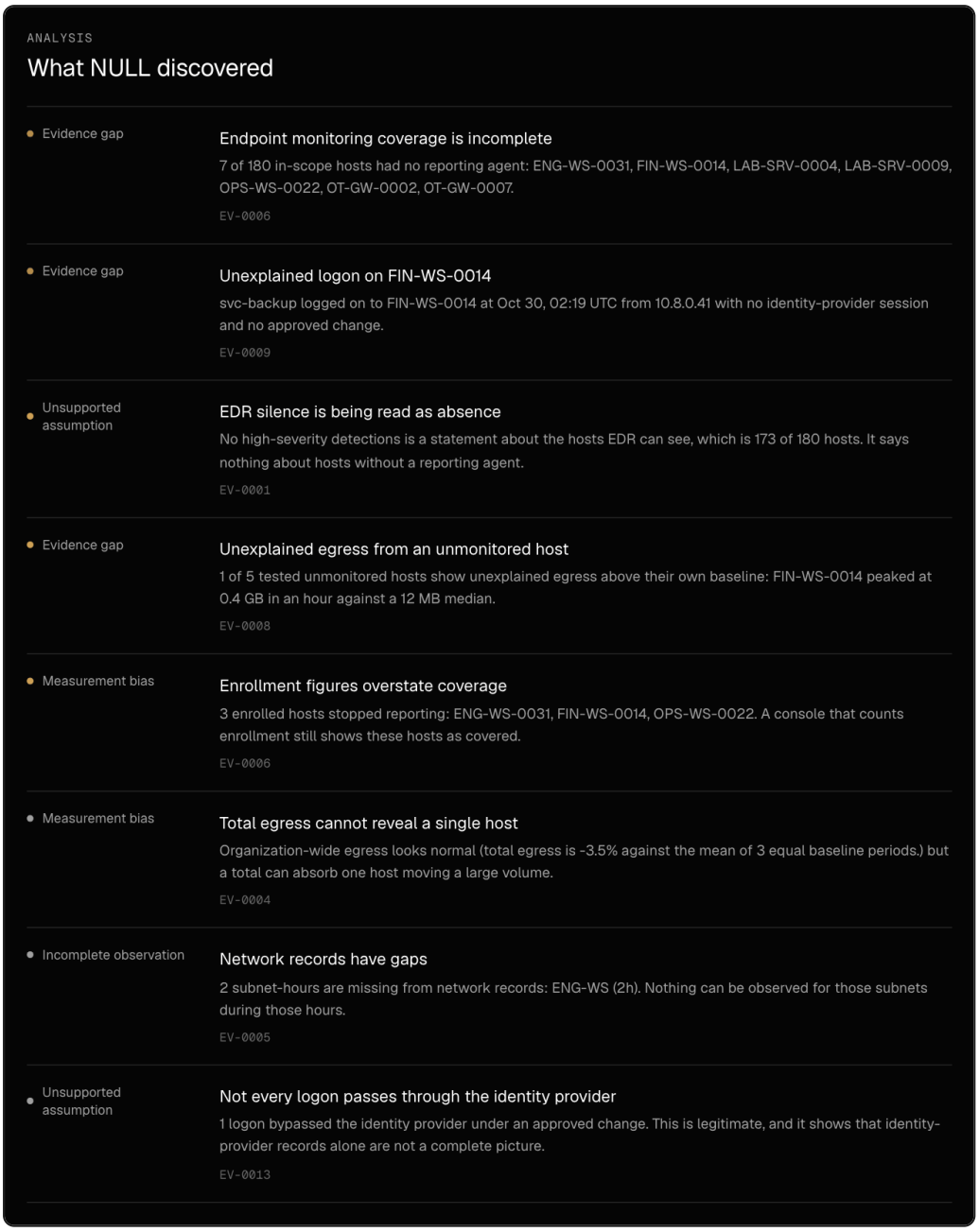


FIGURE 13 Findings, each classified and linked to the evidence behind it.

The conclusion lists the evidence on each side, the state of every assumption, and the limits of the result. Because the demo has a known answer, a button reveals it for comparison. The engine has no access to that answer, and a test enforces this.

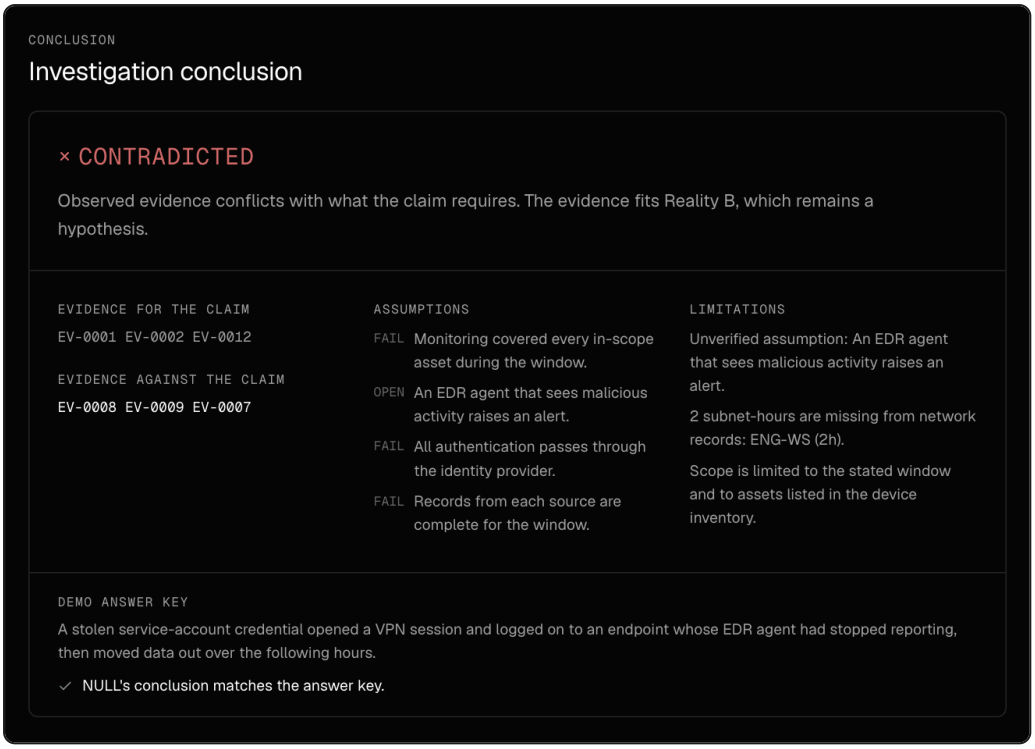


FIGURE 14 Conclusion with the demo answer key revealed. NULL's conclusion matches.

4.5 Evidence Network

The network draws how sources, evidence, assumptions, hypotheses, claims and fixes depend on each other. Edges are typed: supports, contradicts, derived from, depends on, challenges, resolves. Selecting a node dims everything unrelated and explains the relationships in the inspector.

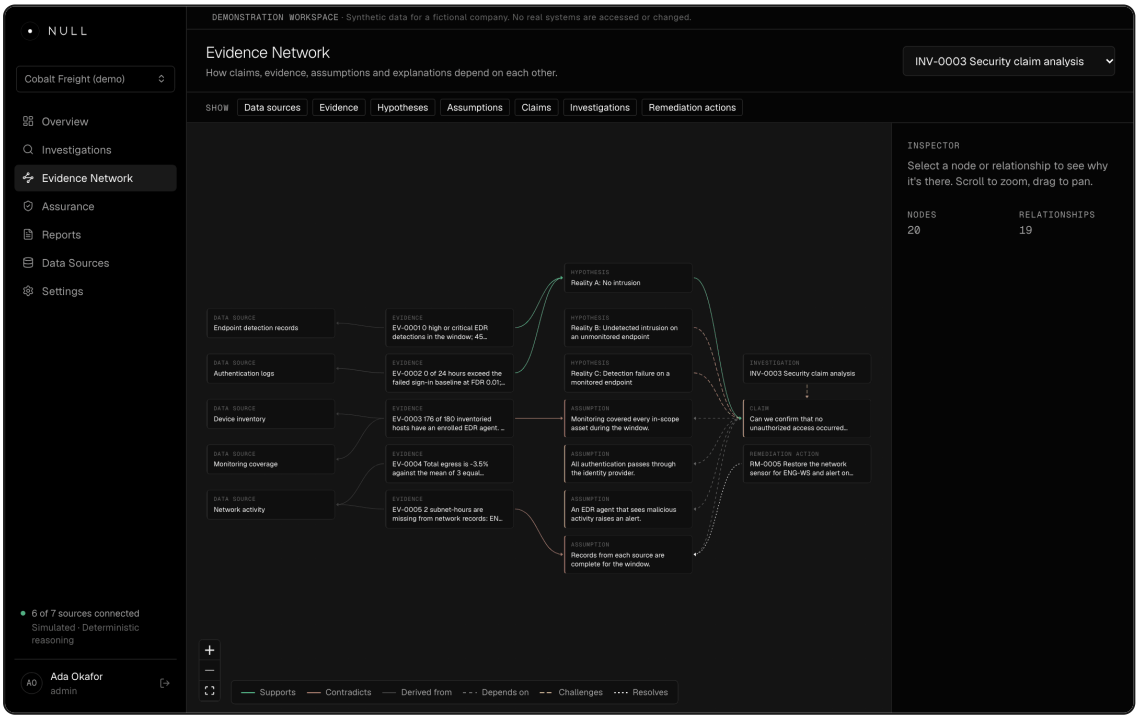


FIGURE 15 Evidence Network for INV-0003: 20 nodes and 19 relationships, with filters above.

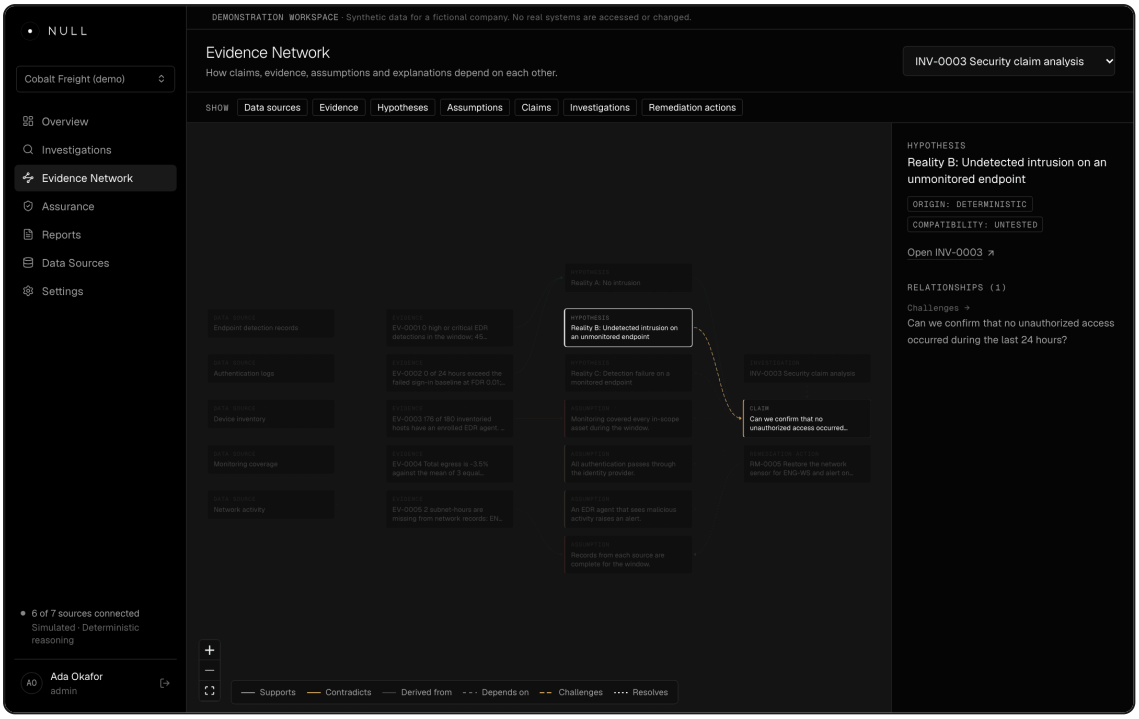


FIGURE 16 Reality B selected. The inspector shows its origin, compatibility and relationships.

4.6 Assurance

Assurance has three parts. **Preventive assurance** is measured from current records. It shows coverage per asset class and separates hosts that were never enrolled from hosts that are enrolled but silent.

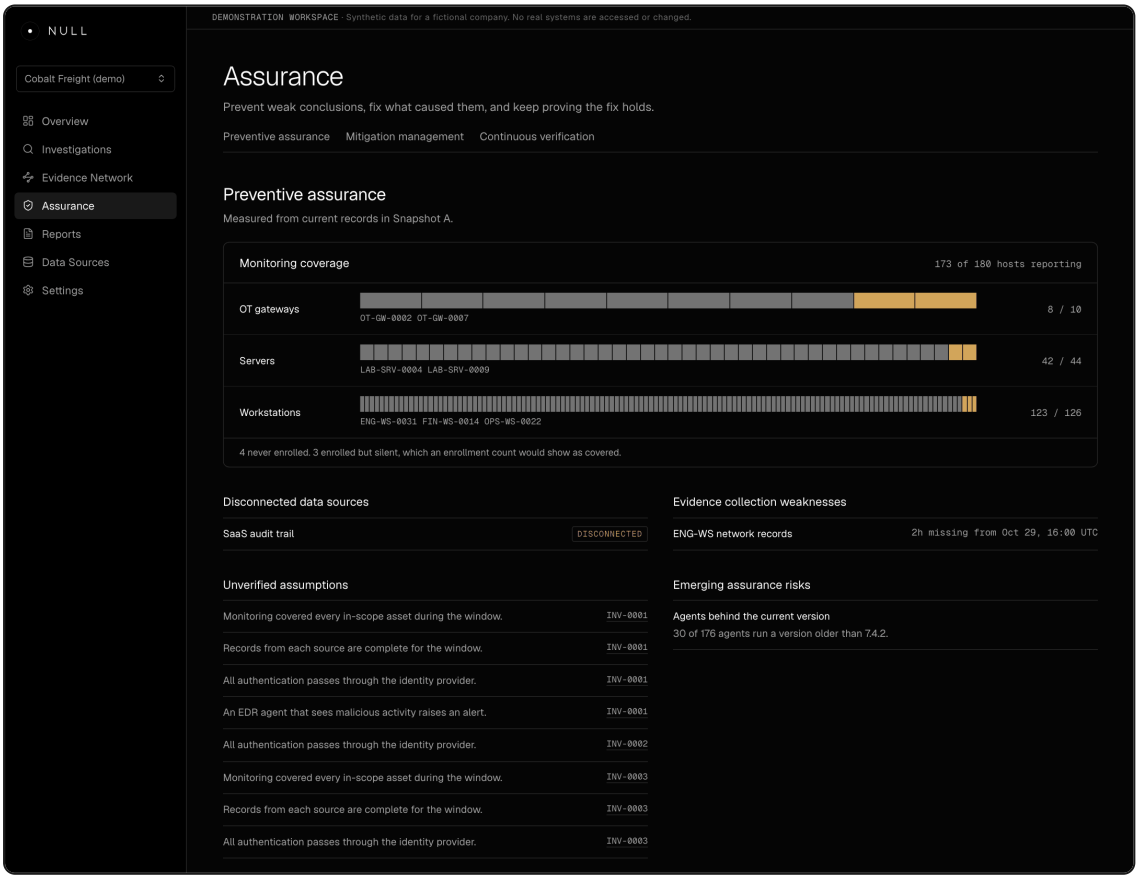


FIGURE 17 Preventive assurance on the seeded demo, with coverage by asset class and open weaknesses.

Mitigation management holds each corrective action, which needs an approval, an execution, and evidence that it worked. Execution is simulated, and the dialog says so.

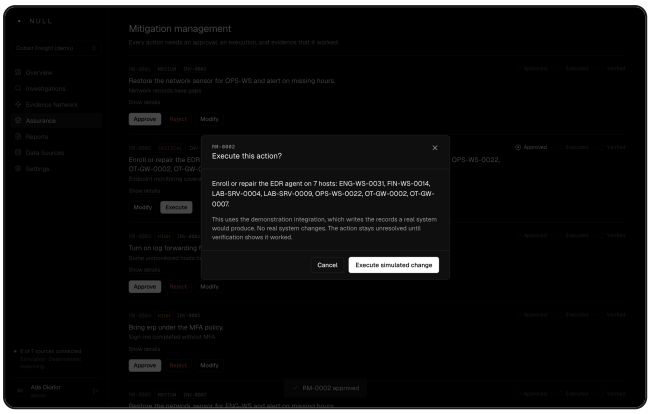


FIGURE 18 Execution confirmation. It uses the demonstration integration, and no real system changes.

Execution alone proves nothing. In the demo the first verification fails on purpose: one gateway never reported, so the action stays unresolved.



FIGURE 19 First verification fails. Only 6 of 7 hosts have a heartbeat after execution, so Verified stays open.

Continuous verification keeps testing fixed weaknesses. Clicking a button can't resolve an action. Only a passing procedure can, and a later run can reopen it.

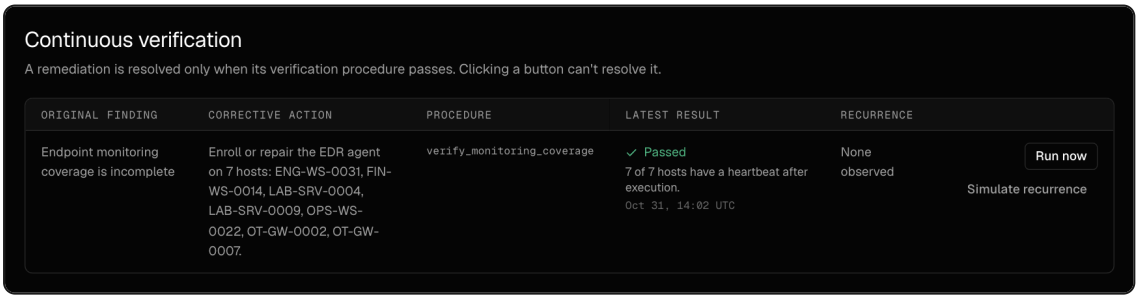


FIGURE 20 After a second execution, verification passes for all 7 hosts.

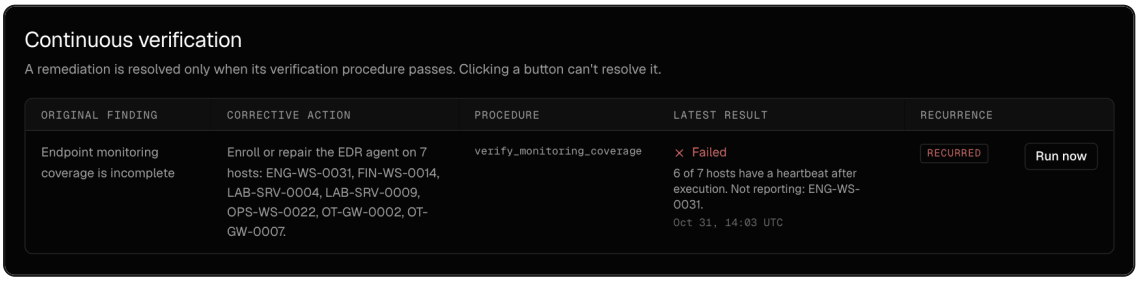


FIGURE 21 A simulated recurrence is caught on the next run and the action is marked RECURRED.

4.7 Reports

A report is a frozen, hashed record of what an investigation established at a point in time, including what remains unknown. It can be printed or exported as PDF, and its evidence as CSV.

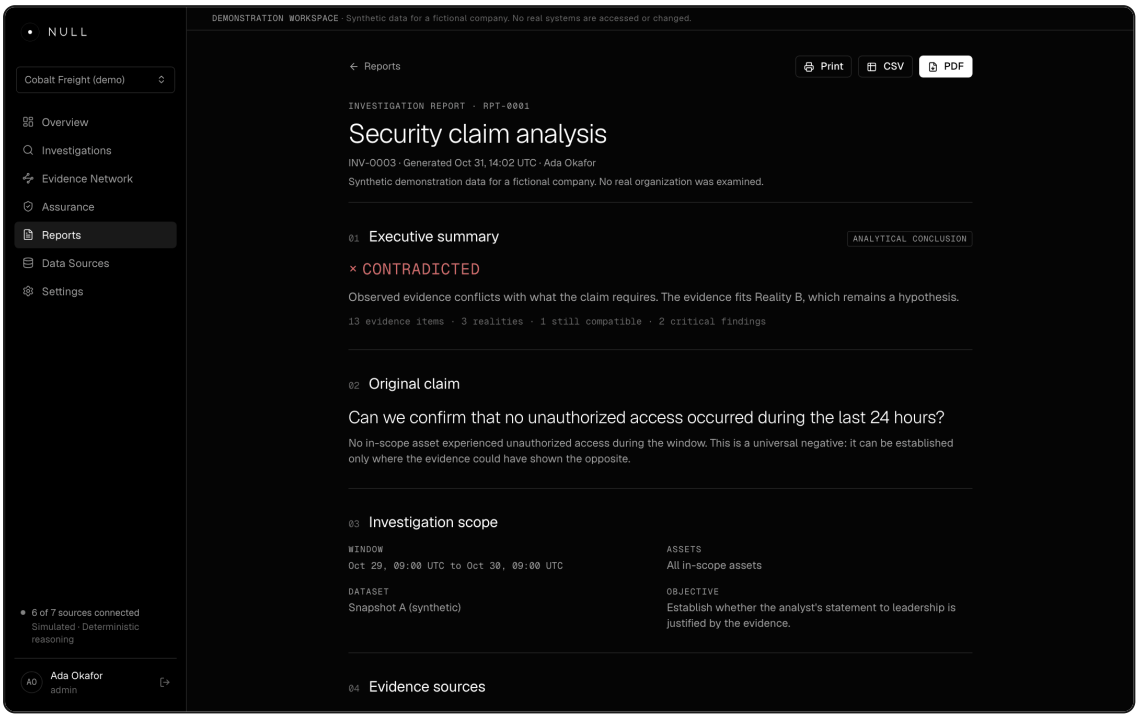
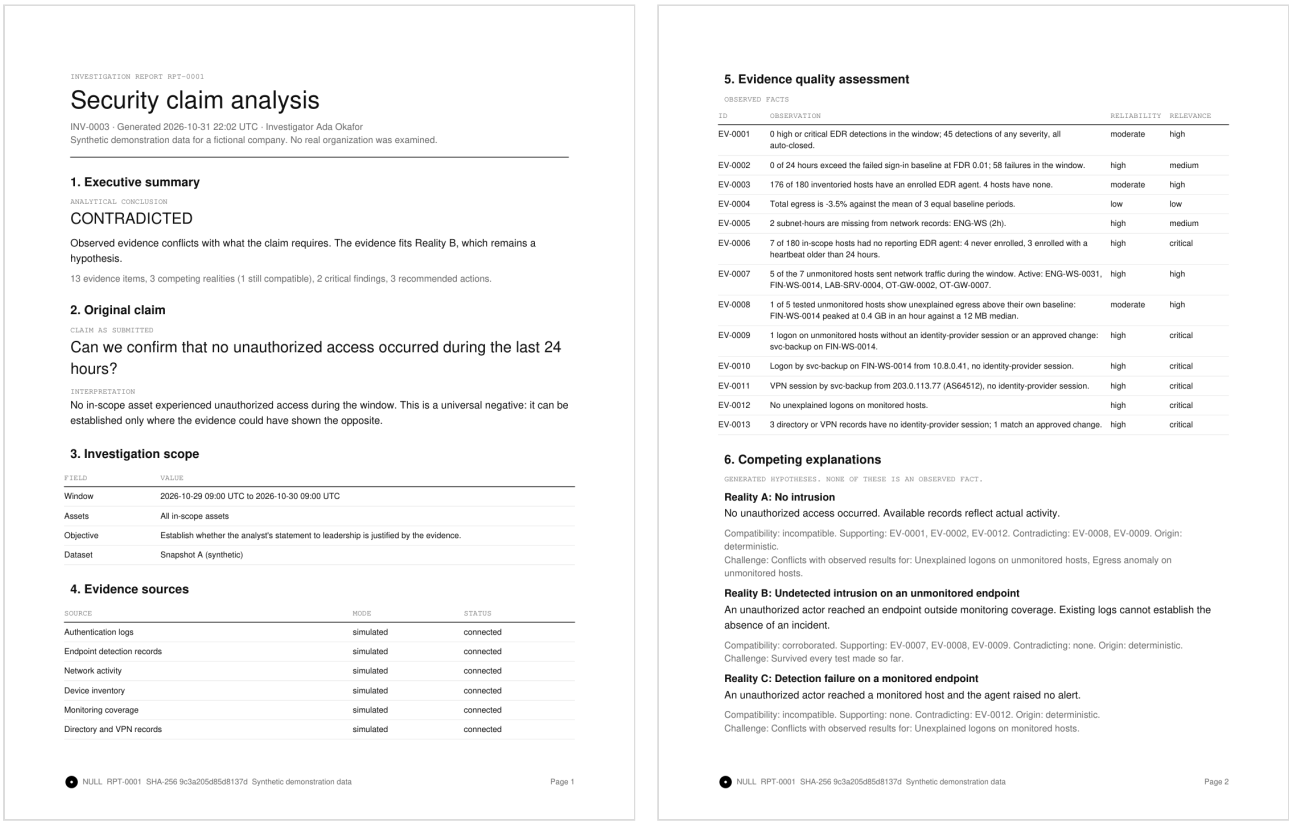


FIGURE 22 Report view. Numbered sections start with the executive summary.



INVESTIGATION REPORT RPT-0001

Security claim analysis

INV-0003 · Generated 2026-10-31 22:02 UTC · Investigator Ada Okafor
Synthetic demonstration data for a fictional company. No real organization was examined.

1. Executive summary

ANALYTICAL CONCLUSION

CONTRADICTED

Observed evidence conflicts with what the claim requires. The evidence fits Reality B, which remains a hypothesis.

13 evidence items, 3 competing realities (1 still compatible), 2 critical findings, 3 recommended actions.

2. Original claim

CLAIM AS SUBMITTED

Can we confirm that no unauthorized access occurred during the last 24 hours?

INTERPRETATION

No in-scope asset experienced unauthorized access during the window. This is a universal negative: it can be established only where the evidence could have shown the opposite.

3. Investigation scope

FIELD	VALUE
Window	2026-10-29 09:00 UTC to 2026-10-30 09:00 UTC
Assets	All in-scope assets
Objective	Establish whether the analyst's statement to leadership is justified by the evidence.
Dataset	Snapshot A (synthetic)

4. Evidence sources

SOURCE	MODE	STATUS
Authentication logs	simulated	connected
Endpoint detection records	simulated	connected
Network activity	simulated	connected
Device inventory	simulated	connected
Monitoring coverage	simulated	connected
Directory and VPN records	simulated	connected

NULL RPT-0001 SHA-256 9c3a205d85d8137d Synthetic demonstration data Page 1

5. Evidence quality assessment

OBSERVED FACTS

ID	OBSERVATION	RELIABILITY	RELEVANCE
EV-0001	0 high or critical EDR detections in the window; 45 detections of any severity, all auto-closed.	moderate	high
EV-0002	0 of 24 hours exceed the failed sign-in baseline at FDR 0.01; 58 failures in the window.	high	medium
EV-0003	176 of 180 inventoried hosts have an enrolled EDR agent. 4 hosts have none.	moderate	high
EV-0004	Total egress is -3.5% against the mean of 3 equal baseline periods.	low	low
EV-0005	2 subnet-hours are missing from network records: ENG-WS (2h).	high	medium
EV-0006	7 of 180 in-scope hosts had no reporting EDR agent: 4 never enrolled, 3 enrolled with a heartbeat older than 24 hours.	high	critical
EV-0007	5 of the 7 unmonitored hosts sent network traffic during the window. Active: ENG-WS-0031, FIN-WS-0014, LAB-SRV-0004, OT-GW-0002, OT-GW-0007.	high	high
EV-0008	1 of 5 tested unmonitored hosts show unexplained egress above their own baseline: FIN-WS-0014 peaked at 0.4 GB in an hour against a 12 MB median.	moderate	high
EV-0009	1 logon on unmonitored hosts without an identity-provider session or an approved change: svc-backup on FIN-WS-0014.	high	critical
EV-0010	Logon by svc-backup on FIN-WS-0014 from 10.8.0.41, no identity-provider session.	high	critical
EV-0011	VPN session by svc-backup from 203.0.113.77 (AS64512), no identity-provider session.	high	critical
EV-0012	No unexplained logons on monitored hosts.	high	critical
EV-0013	3 directory or VPN records have no identity-provider session; 1 match an approved change.	high	critical

6. Competing explanations

GENERATED HYPOTHESES, NONE OF THESE IS AN OBSERVED FACT.

Reality A: No intrusion

No unauthorized access occurred. Available records reflect actual activity.

Compatibility: incompatible. Supporting: EV-0001, EV-0002, EV-0012. Contradicting: EV-0008, EV-0009. Origin: deterministic.
Challenge: Conflicts with observed results for: Unexplained logons on unmonitored hosts, Egress anomaly on unmonitored hosts.

Reality B: Undetected intrusion on an unmonitored endpoint

An unauthorized actor reached an endpoint outside monitoring coverage. Existing logs cannot establish the absence of an incident.

Compatibility: corroborated. Supporting: EV-0007, EV-0008, EV-0009. Contradicting: none. Origin: deterministic.
Challenge: Survived every test made so far.

Reality C: Detection failure on a monitored endpoint

An unauthorized actor reached a monitored host and the agent raised no alert.

Compatibility: incompatible. Supporting: none. Contradicting: EV-0012. Origin: deterministic.
Challenge: Conflicts with observed results for: Unexplained logons on monitored hosts.

NULL RPT-0001 SHA-256 9c3a205d85d8137d Synthetic demonstration data Page 2

FIGURE 23 First two pages of the exported PDF. The timestamp defect in Section 9 is visible in the header line.

4.8 Data Sources

The page opens by stating that no live integrations are connected. Each of the seven sources is badged as simulated and shows its real record count, freshness, coverage and permissions. A disconnected source is shown as disconnected. Disconnecting one is allowed, and NULL then reports the affected checks as blocked.

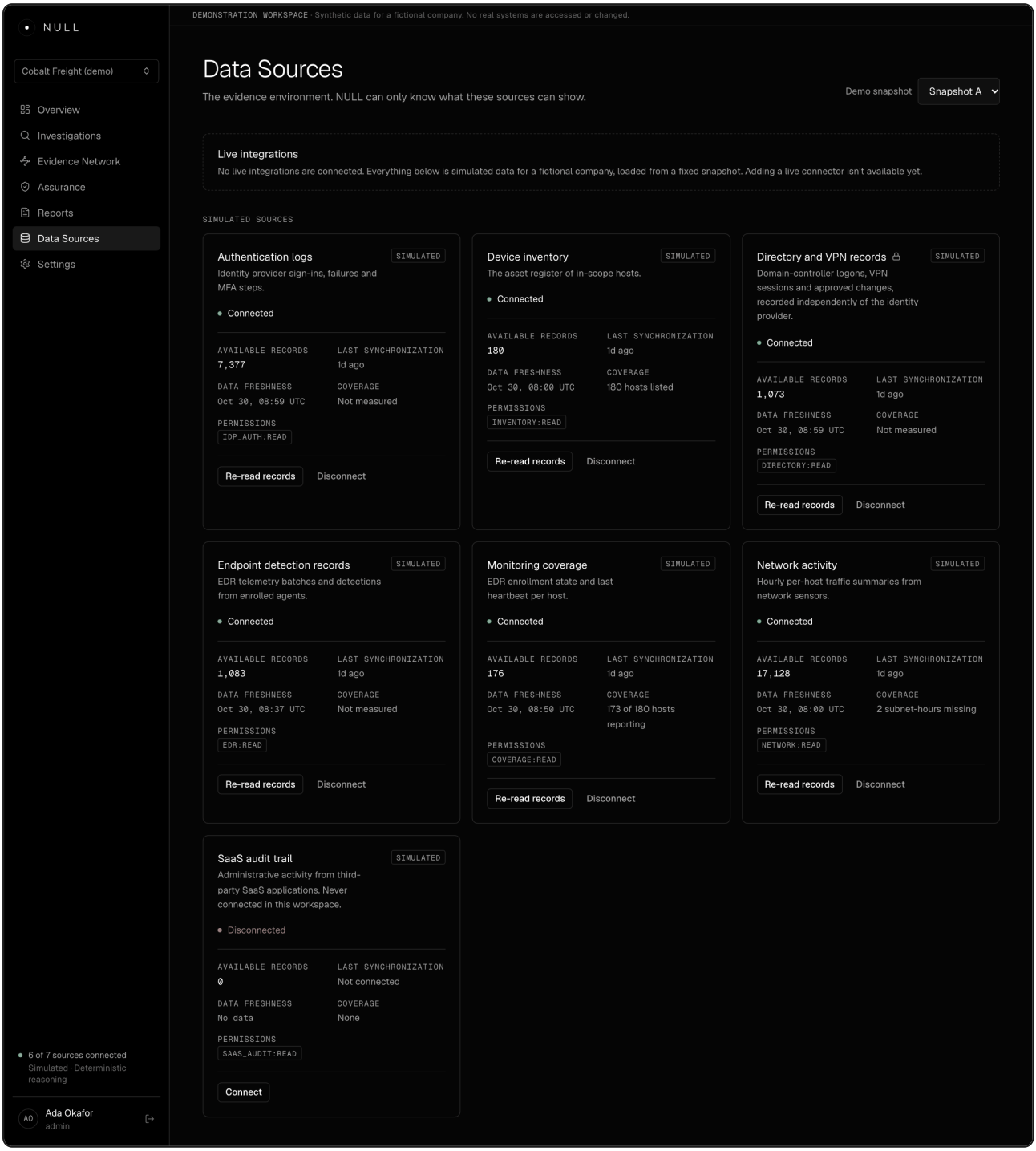


FIGURE 24 Data Sources. Six of seven simulated sources are connected. The directory source is restricted.

4.9 Settings

Settings covers Workspace, Profile, Team, Roles, API, AI, Audit logs, Notifications and Security. The AI tab shows plainly that NULL is running on deterministic reasoning and why. The audit log records sign-ins, approvals, executions and other actions.

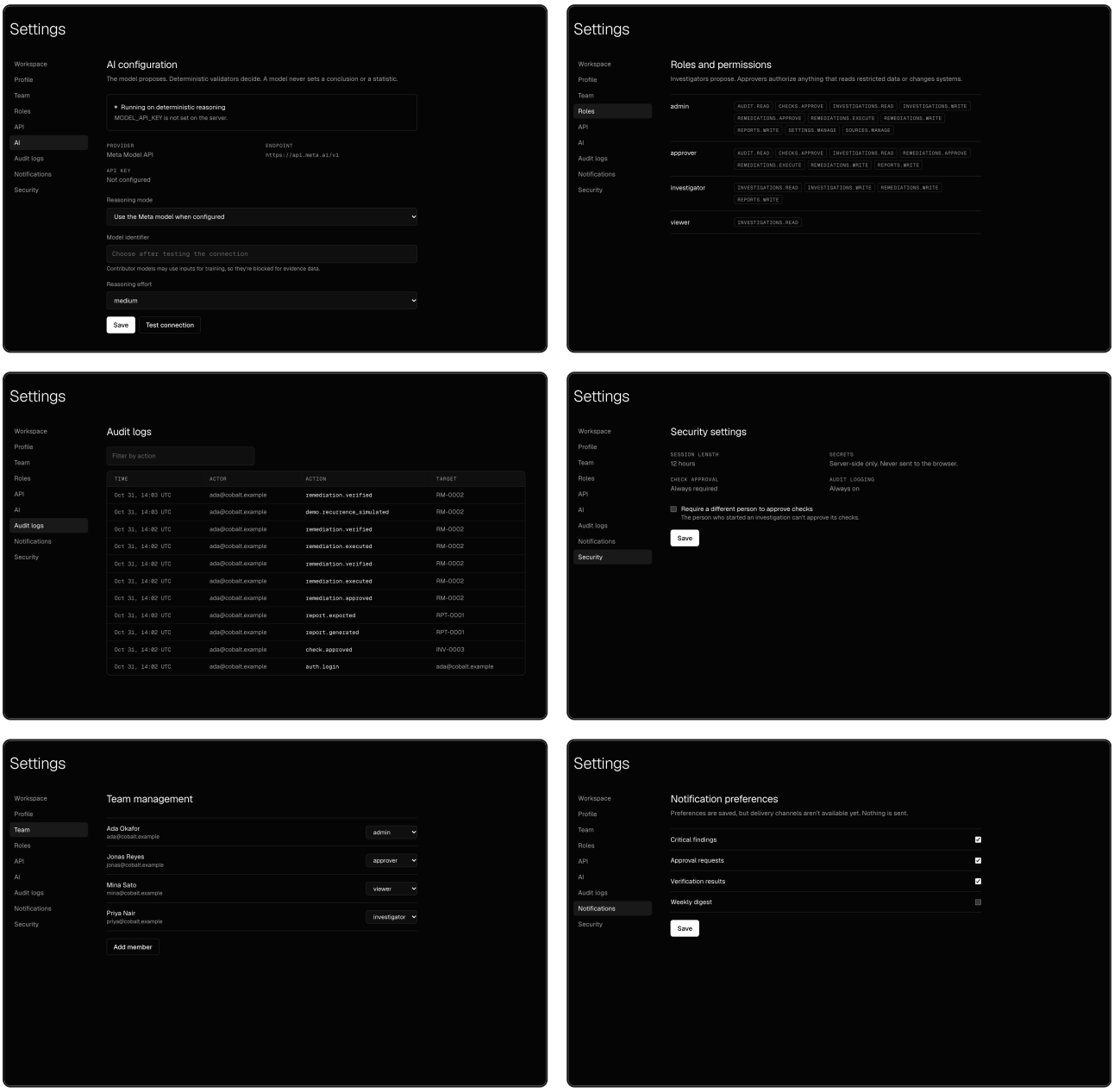


FIGURE 25 Settings tabs: AI, Roles, Audit logs, Security, Team and Notifications.

4.10 Phone layout

The app works at 390 px wide. The sidebar becomes a scrolling tab bar, and no page scrolls sideways.

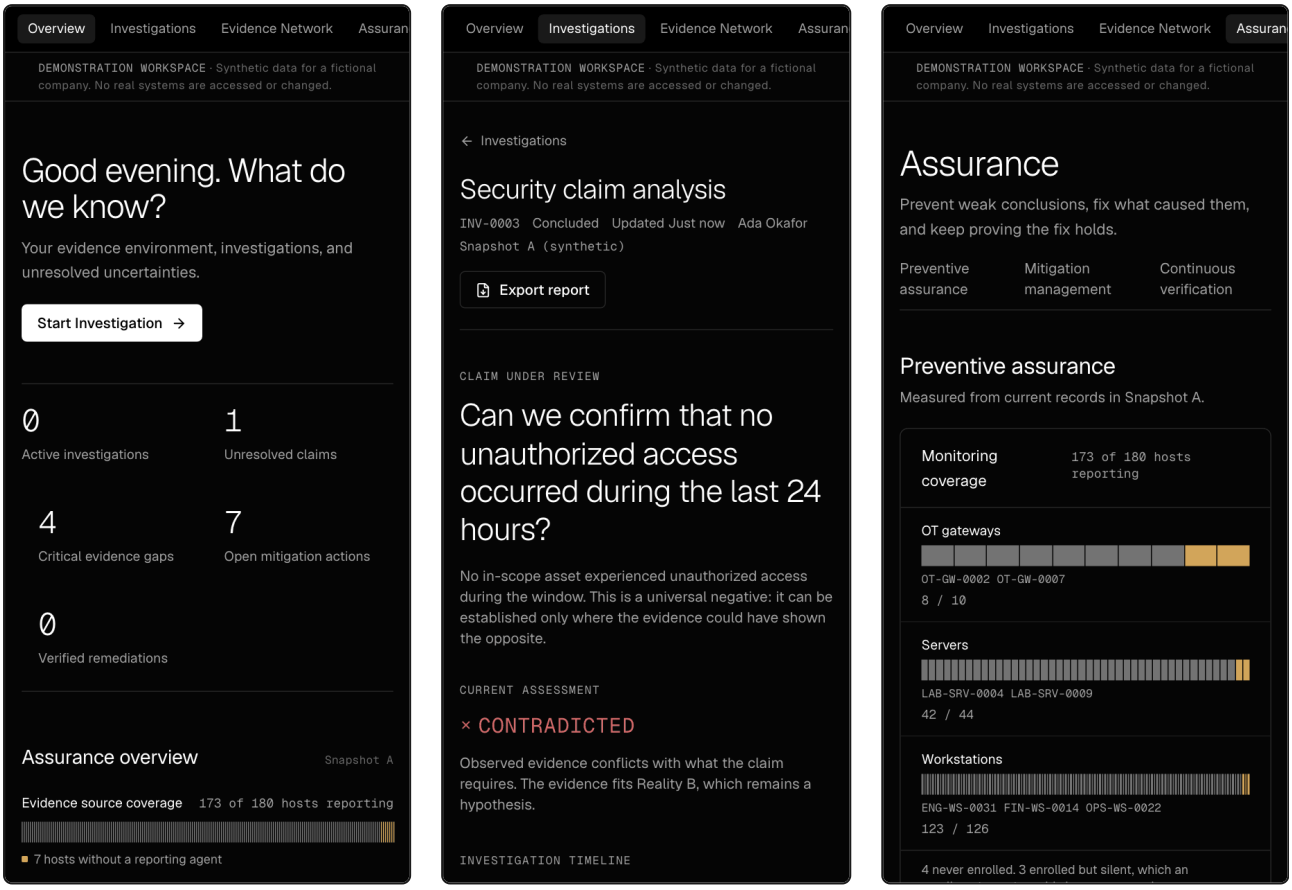


FIGURE 26 Overview, workspace and Assurance at phone width.

5. Demonstration data

The demo uses four seeded snapshots of one fictional company, each with a different hidden outcome. The same claim on the same code gives a different conclusion, which shows the result comes from the evidence rather than from a script.

SNAPSHOT	WHAT ACTUALLY HAPPENED	NULL CONCLUDES	RECORDS
A	Intrusion on an unmonitored endpoint	CONTRADICTED	27,017
B	No intrusion, but two gateways have no evidence at all	PARTIALLY IDENTIFIED	27,182
C	No intrusion, complete coverage	SUPPORTED	27,235
D	Intrusion on a monitored endpoint, no EDR alert	CONTRADICTED	27,405

All data covers a fixed window ending 30 October 2026 at 09:00 UTC. The answer key lives in a separate table that the analysis engine can't reach.

Seeded investigations

ID	NAME	SNAPSHOT	SEEDED ASSESSMENT	STATUS
INV-0001	Weekly access assurance	B	PARTIALLY IDENTIFIED	Concluded
INV-0002	MFA enforcement review	A	CONTRADICTED	Concluded
INV-0003	Security claim analysis	A	UNDERDETERMINED	Awaiting approval

Demo accounts

EMAIL	ROLE	CAN DO
ada@cobalt.example	admin	Everything, including sources and settings
jonas@cobalt.example	approver	Approve checks, approve and execute fixes, read the audit log
priya@cobalt.example	investigator	Run investigations, propose fixes, write reports
mina@cobalt.example	viewer	Read investigations only

All four use the password set by `NULL_DEMO_PASSWORD` in `.env`. It is not stored in the repository.

6. Architecture

A Next.js frontend talks to a FastAPI backend through its own origin, so the browser never sees the backend or its secrets. PostgreSQL holds raw records and investigation state.

LAYER	TECHNOLOGY
Frontend	Next.js 16.2.3, React 19.2.4, TypeScript, Tailwind CSS 4, React Flow (`@xyflow/react`), Geist
Backend	Python, FastAPI, SQLAlchemy 2, Alembic
Analysis	pandas, NumPy, SciPy, statsmodels
Database	PostgreSQL 14 or later, 19 application tables
Security	Argon2 password hashes, JWT in an HTTP-only cookie, role-based permissions, audit log
Reports	ReportLab for PDF export
Model provider	Meta Model API, optional, with the model ID set by configuration
Tests	pytest and Playwright

Repository layout

PATH	CONTENTS
`backend/app/analysis`	Deterministic engine: compiler, observables, realities, validator, narrative
`backend/app/reasoning`	Provider interface, Meta client, rule-based roles
`backend/app/services`	Orchestration, graph, remediation, assurance, reports, sources
`backend/app/api`	FastAPI routers
`backend/app/demo`	Synthetic data generator and seed
`frontend/src/app`	Next.js pages
`frontend/scripts`	Logo generator, writes `public/brand`

Data model

AREA	TABLES
Workspace	`workspaces`, `users`, `audit_logs`
Evidence	`data_sources`, `evidence_records`, `evidence_items`, `dataset_ground_truth`
Investigation	`investigations`, `timeline_events`, `tool_executions`, `hypotheses`, `assumptions`, `findings`, `recommended_checks`
Assurance	`remediations`, `verification_runs`
Graph and reports	`graph_nodes`, `graph_edges`, `reports`

API

The API has 41 routes across eight routers under `/api/v1`. Interactive documentation is at `/api/docs` outside production.

ROUTER	ROUTES	PURPOSE
`auth`	3	Sign in, sign out, current user
`overview`	1	Overview page data
`investigations`	9	Create and read investigations, compile claims, read evidence, approve or reject checks, answer key
`network`	1	Evidence graph
`assurance`	7	Assurance data, plus approve, reject, modify, execute and verify fixes, and simulate recurrence
`reports`	5	Generate and read reports, PDF and CSV export
`sources`	5	List, sync, connect and disconnect sources, switch snapshot
`settings`	10	Workspace, profile, AI, notifications, security, members, audit log

Roles and permissions

PERMISSION	ADMIN	APPROVER	INVESTIGATOR	VIEWER
`investigations.read`	Yes	Yes	Yes	Yes
`investigations.write`	Yes	No	Yes	No
`checks.approve`	Yes	Yes	No	No
`remediations.write`	Yes	Yes	Yes	No
`remediations.approve`	Yes	Yes	No	No
`remediations.execute`	Yes	Yes	No	No
`reports.write`	Yes	Yes	Yes	No
`audit.read`	Yes	Yes	No	No
`sources.manage`	Yes	No	No	No
`settings.manage`	Yes	No	No	No

7. Running NULL

You need Node 20 or later, `uv` and PostgreSQL 14 or later.

```
cp .env.example .env     # set NULL_SECRET_KEY and NULL_DEMO_PASSWORD
createdb null_app
make setup
make seed
make dev                 # http://localhost:3000
```

Sign in as `ada@cobalt.example`. If `NULL_DEMO_PASSWORD` was left empty, `make seed` prints a generated one. Without a local PostgreSQL, run `docker compose up -d` and set `NULL_DATABASE_URL=postgresql+psycopg://null:null_local_only@localhost/null_app`.

COMMAND	WHAT IT DOES
`make setup`	Installs backend and frontend dependencies, and creates `.env` if missing
`make seed`	Runs migrations and loads the demo
`make reset`	**Drops every record**, including your own investigations, and rebuilds the demo
`make dev`	Starts the API on port 8000 and the web app on port 3000
`make test`	Runs backend tests on a separate test database
`make e2e`	Runs browser tests. Needs both servers, and **wipes the demo data** before and after
`make smoke-meta`	Calls the real Meta Model API. Needs `MODEL_API_KEY` and `META_MODEL_ID`

Connecting the Meta model

Set `MODEL_API_KEY` and `META_MODEL_ID` in `.env`, or choose the model in Settings > AI, then run `make smoke-meta`. Model IDs ending in `-contributor` may be used to train Meta's models, so they are blocked unless `NULL_ALLOW_CONTRIBUTOR_MODELS=true`.

Running the demonstration

1. Open **Security claim analysis** from Overview. Its claim is UNDERDETERMINED.
2. In **Competing realities**, see which observations would separate the explanations.
3. Select **Review Investigation** and approve. The claim becomes CONTRADICTED. Reveal the answer key to compare.
4. Open **Assurance**. Approve, execute and verify the agent enrollment. The first verification fails, so execute again.
5. Select **Simulate recurrence** and verify again to see the weakness caught.

8. Verification status

These checks were run on 31 October 2026 against version `ef9ad5e`, on a separate copy of the app and a fresh database.

CHECK	RESULT
Backend tests: ground truth, validator, Meta client, API, permissions	82 of 82 passed
Browser tests on the production build: full demo, new investigation, network, viewer role, keyboard, phone width	8 of 8 passed
Python lint (`ruff`)	Clean
Server errors during the browser test and capture runs	0 tracebacks
Conclusions for snapshots A to D match the answer key	Covered by backend tests
Live Meta Model API (`make smoke-meta`)	Not run: no API key
`npm audit` dependency review	Not reviewed

The Meta client is tested only against a fake server that enforces the documented API rules, so the live integration remains unproven until `make smoke-meta` passes.

9. Known limits and defects

9.1 Defects found while preparing this document

None of these change a conclusion. The first matters most, because a report is meant to serve as a record.

SEVERITY	DEFECT	WHERE
High	**Exported PDF times can be wrong.** Times are printed with a “UTC” label but keep the database session's time zone. On a server in Asia/Manila, a report created at 14:02 UTC prints as 22:02 UTC. Time zones with no offset hide the problem.	`backend/app/services/reports.py:305, :369`
Medium	**Evidence IDs vary between runs.** Approved checks are loaded without a sort order, so the same investigation can number its later evidence differently. Separate runs produced EV-0006 and EV-0008 for the same logon evidence.	`backend/app/services/orchestrator.py:735`
Medium	**Audit log hides verification results.** Every verification run is listed as `remediation.verified`, including failed runs and caught recurrences, so the log reads as if each one passed. The result is stored with the entry, but the page does not show it.	`backend/app/api/assurance.py:195`
Low	**Mixed time zones in the evidence drawer.** Raw records show the server's offset (+08:00), while the rest of the page shows UTC.	Evidence drawer
Low	**Provenance column clipped.** At 1440 px the evidence table is 920 px wide in an 834 px area, so the last column needs a sideways scroll.	Investigation workspace
Low	**Misaligned tiles on phones.** Only the first Overview tile drops its left padding, so the rows below sit further right.	`frontend/src/app/(app)/page.tsx:66`

9.2 Before real use

AREA	CURRENT LIMIT
Connectors	None exist. All sources are simulated, and execution writes records instead of changing systems. Each connection needs a reader and an execution integration.
Model integration	Never exercised against the live Meta API.
Claim types	Three only. Adding one means adding observables, realities and a compiler rule.
Background work	Analysis runs in a thread inside the API process, and a restart marks running investigations as failed. Several workers would need a real queue.
Login throttling	Kept in memory, so it applies per process.
Data at rest	Raw evidence sits in PostgreSQL, unencrypted beyond what the database provides. There is no retention policy.
Access	No personal API tokens and no multi-workspace support.
Notifications	Preferences are saved, but nothing is sent.
Production mode	Set `NULL_ENV=production` so the server requires a secret, marks cookies Secure and hides the API docs.